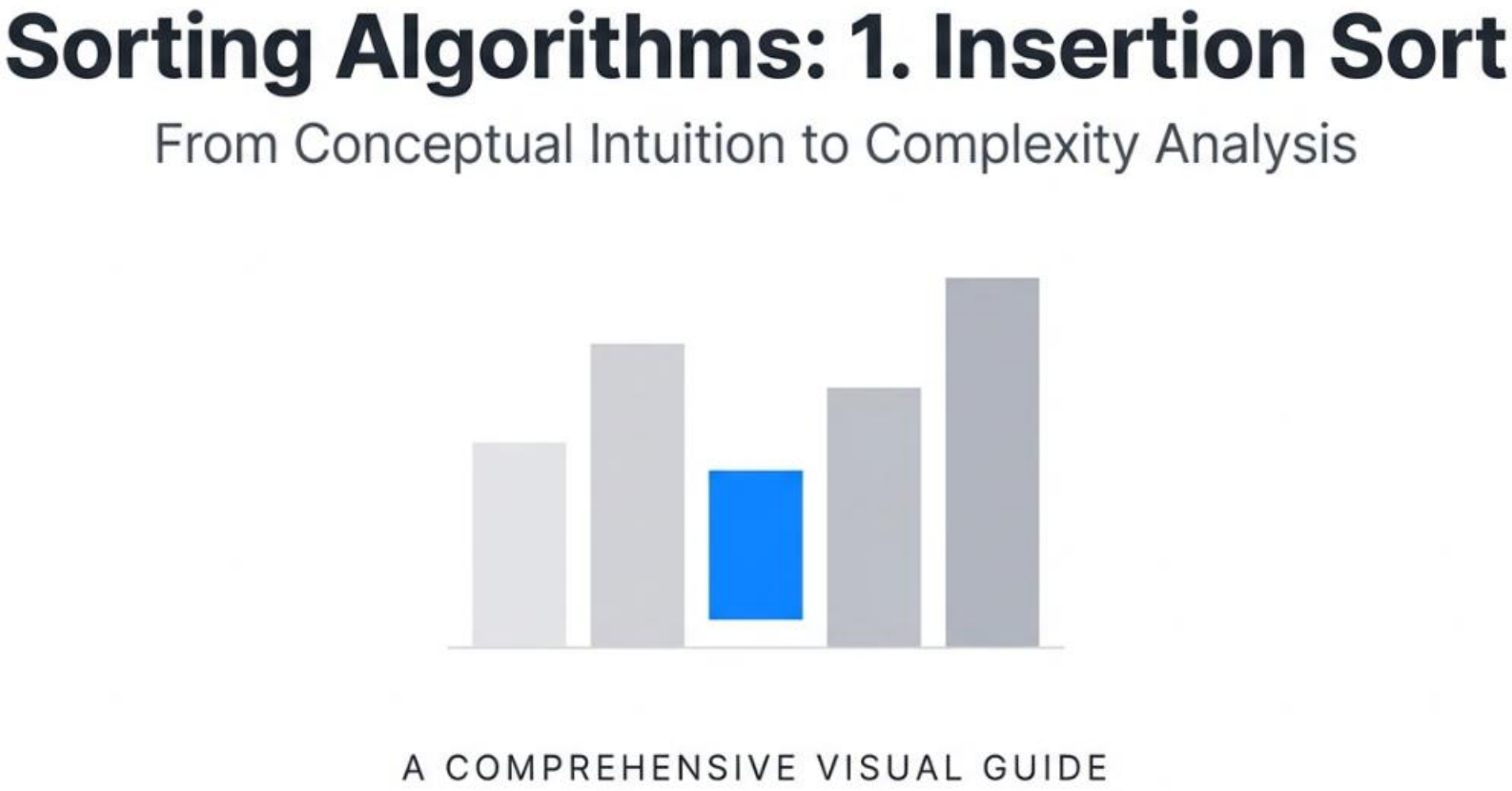
Lecture 7

Sorting Algorithms: Insertion, Selection



© 2026 Ahmed Elrefaiy

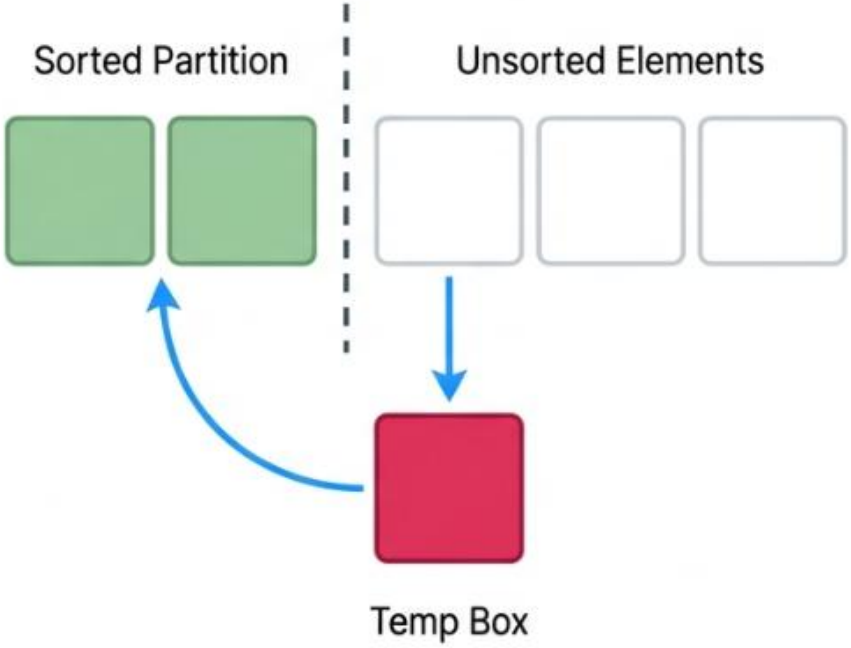
The Core Idea: Build the Sort, One Element at a Time

Insertion Sort continuously maintains a "sorted partition," inserting new elements into their correct relative positions—much like sorting a hand of playing cards.

The Logic Loop:

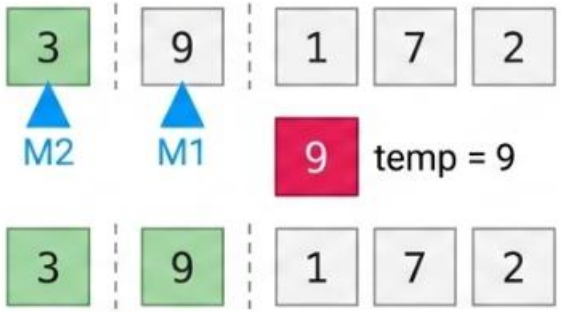
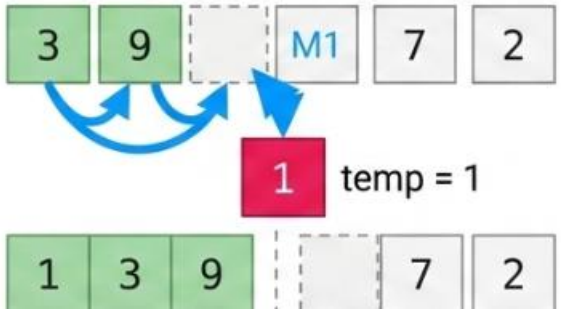
- **Start at Element 2**: The first element is inherently in its own sorted partition.
- **Isolate & Store:** Extract the next unsorted element into a temporary variable (temp).
- **Compare & Shift:** Compare temp against the sorted elements from right to left. If a sorted element is larger, shift it one space right.
- **Insert:** Place temp into the newly opened space once a smaller element is found.

Key Insight: By shifting larger elements to the right rather than immediately swapping, we efficiently create an empty slot for the target element.

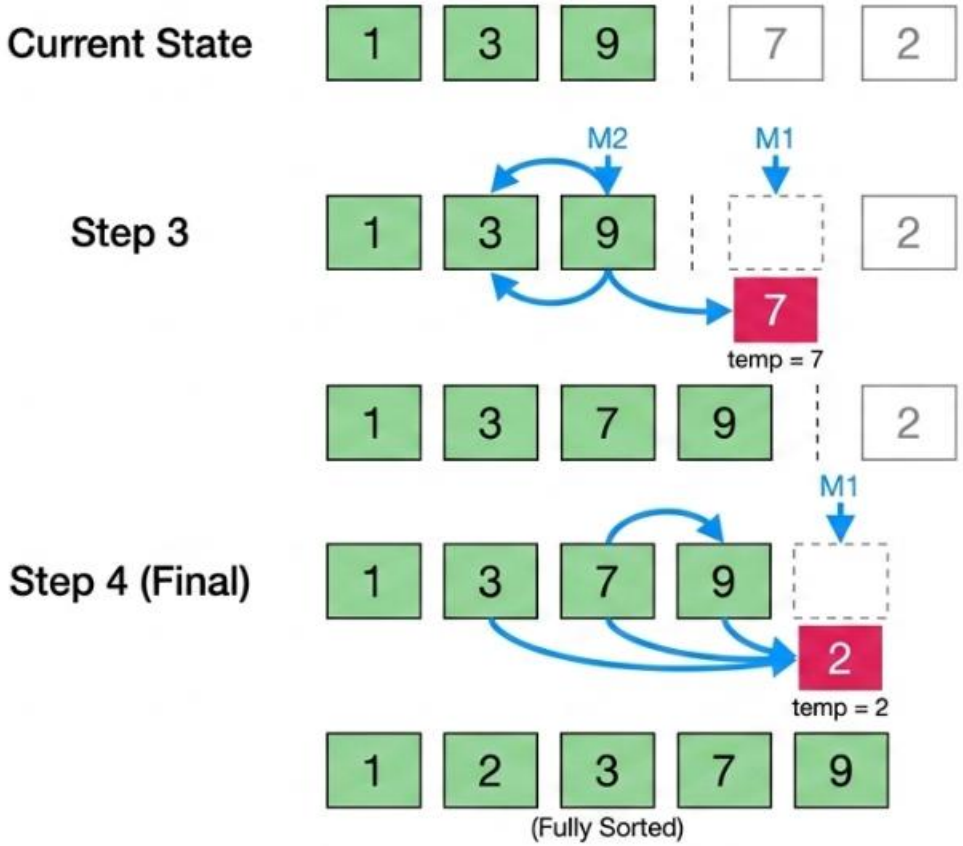


Tracing the Algorithm: Initial Passes

Goal: Sort the array `[3, 9, 1, 7, 2]`

Initial State	*Note: First element (3) is naturally sorted.*	
Step 1		**Extract:** temp = 9 **Compare:** M2 (3) < temp (9)? No shifts needed. 9 stays in place.
Step 2		**Extract:** temp = 1 **Compare & Shift:** 1 < 9 (Shift 9 right), 1 < 3 (Shift 3 right). **Insert:** Place temp (1) at Index 0.

Tracing the Algorithm: Finalizing the Sort



Extract: temp = 7
Compare & Shift: 7 < 9 (Shift 9 right). 7 > 3 (Stop).
Insert: Place temp (7) in the open slot after 3.

Extract: temp = 2
Compare & Shift: 2 < 9, 2 < 7, 2 < 3 (Shift all right). 2 > 1 (Stop).
Insert: Place temp (2) after 1.

Formalizing the Logic: The Algorithm

```
function insertionSort(arr, n):
    for M1 from 1 to n - 1:
        temp = arr[M1]
        M2 = M1 - 1

        while M2 >= 0 and temp < arr[M2]:
            arr[M2 + 1] = arr[M2]
            M2 = M2 - 1

        arr[M2 + 1] = temp
```

The Outer Loop: Iterates through unsorted elements. M1 skips index 0, as a single element is inherently sorted.

The Holding Zone: Safeguards the target element so it is not overwritten during shifting.

The Inner Loop: Runs backward from M1 - 1. Triggers only if the target (temp) is smaller than the evaluated element.

The Shift: Moves larger elements one position to the right, naturally creating a gap.

The Insertion: Places the target into the exact gap created by the shifting process.

Time Complexity Analysis: O, Ω, Θ

Performance depends entirely on how many times the inner while loop (the shifting mechanism) is forced to execute.

Best Case: Ω(N)	**Average Case: Θ(N^2)**	**Worst Case: $O(N^2)$**
Scenario: Array is already sorted (e.g., [1, 2, 3, 7, 9]). **Why:** The while condition (`temp < arr[M2]`) fails immediately on every pass. The inner loop never runs. Zero shifts occur.	**Scenario:** Elements are in random/mixed order. **Why:** On average, half the elements in the sorted partition are shifted for every new element. The nested loops result in **quadratic** scaling.	**Scenario:** Array is reverse sorted (e.g., [9, 7, 3, 2, 1]). **Why:** Every new element is smaller than all previously sorted elements. The inner loop must traverse all the way to index 0 **every time**, maximizing shifts.

Mathematical Insight: In the worst case, the outer loop runs N-1 times. The inner loop runs up to M1 times.

Summation: $\sum$ (from i=1 $\sum_{i=1}^{N-1} i = \frac{N(N-1)}{2} = \frac{1}{2}N^2 - \frac{1}{2}N$. Removing constants yields $O(N^2)$.

Space Complexity: In-Place Sorting

When evaluating Space Complexity, we differentiate between **Input Space** (memory holding the array itself) and **Auxiliary Space** (extra memory required by the algorithm).

Input Space (N Elements)

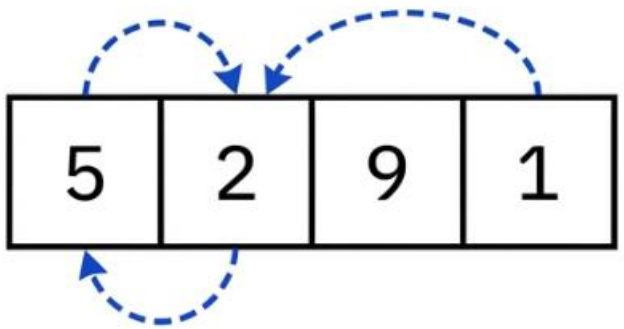
Auxiliary Space

✓ In-Place Sorting

Algorithm Footprint	Conclusion: O(1) Auxiliary Space
To perform Insertion Sort, we define only **three** variables inside the function, regardless of how massive the input array is: M1 (outer loop), M2 (inner traversal), and temp (holding variable).	• Fixed variables (3) mean the extra memory required is constant. • The elements are rearranged entirely within the original array's boundaries. • No new arrays or data structures are created.

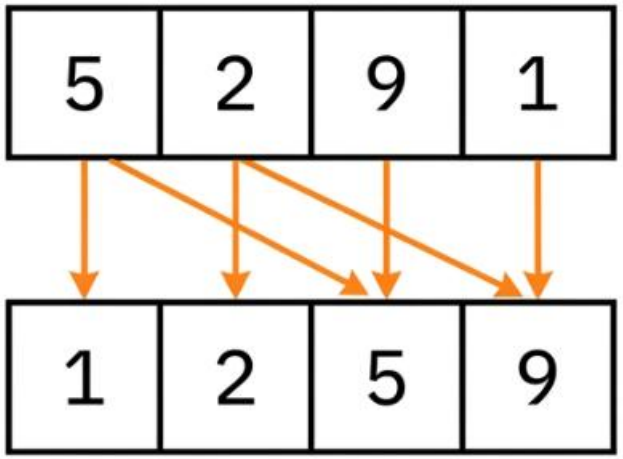
Spatial Architecture: In-Place vs. Out-of-Place

`[ O(1) Extra Space ]`



Self-contained: Modifies the original sequence directly. No auxiliary data structures are required.

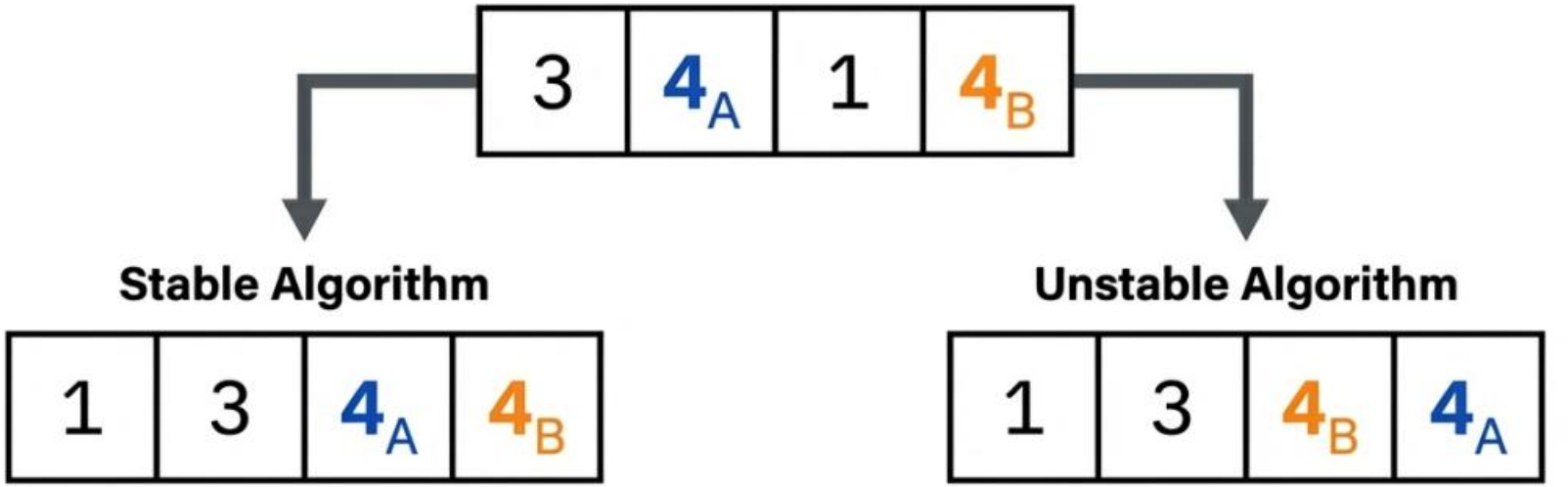
`[ O(N) Extra Space ]`



Requires an intermediate secondary array. Allocates additional memory proportional to the input size.

Relative Order: The Mechanics of Stability

How does the algorithm handle identical values during the sorting process?

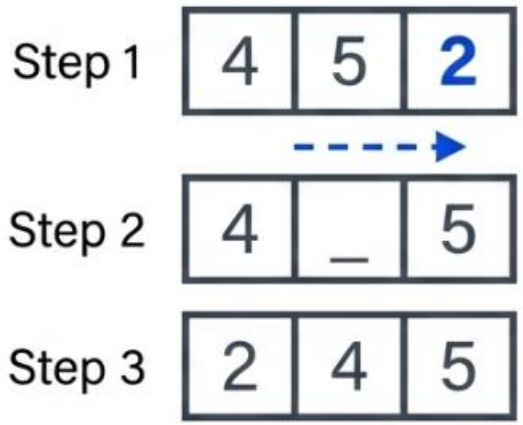


Original relative order is preserved. 4_A appeared first in the input, so it remains first in the output.

Original relative order is reversed or randomized during the sorting mechanics.

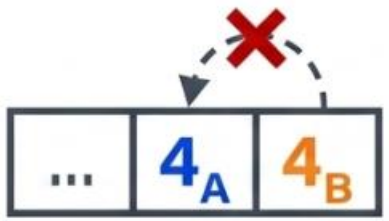
Algorithm Profile: Insertion Sort

Insertion Sort is structurally **In-Place** and naturally **Stable.**



Why is it In-Place?

Elements are shifted linearly to the right **strictly within the original array boundaries.**
A secondary array is never initialized.



Why is it Stable?

The algorithm only shifts elements that are strictly greater than the active target. When **4B** encounters **4A**, the shift halts immediately, perfectly locking locking their original sequence.

SORTING ALGORITHMS SERIES

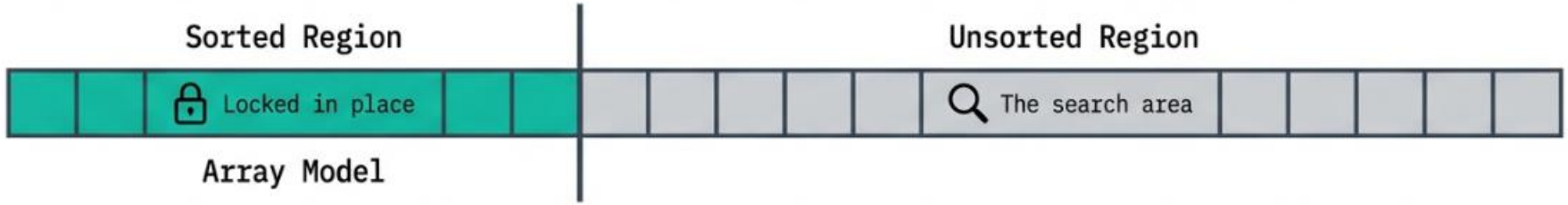
02. Selection Sort

A comprehensive breakdown: Idea, Execution, Complexity, and Properties

Brute-Force Approaches in Computer Science

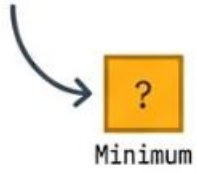
The ‘Brute Force’ Intuition

Finding order through a naive, yet highly systematic mindset.



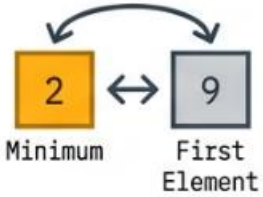
Step 1: Scan & Find.

Search the entire **Unsorted Region** to find the absolute smallest element. Record its index as the **Minimum**.



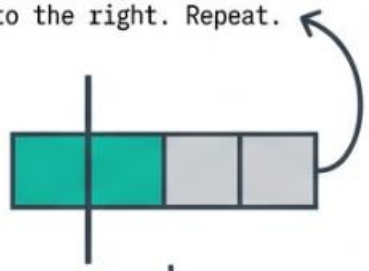
Step 2: Swap.

Move this smallest element to the front, swapping it with the first element of the Unsorted Region.



Step 3: Expand.

Shift the boundary of the **Sorted Region** one space to the right. Repeat.



Why ‘Brute Force’?

It relies on exhaustive searching rather than advanced logic.
It asks every single remaining number,
“Are you the smallest?”

Tracing the Execution (Pass 1 & 2)

Initial Array: [3, 9, 7, 1, 2]

Pass 0	3 9 7 1 2	Array is entirely unsorted. Target is index 0.
Pass 1	3 9 7 1 2 | 1 9 7 3 2	Min value is 1. Swap 3 and 1.
Pass 2	1 | 9 7 3 2 | 1 2 | 7 3 9	Search [9, 7, 3, 2]. Min value is 2. Swap 9 and 2.

Tracing the Execution (Pass 3 & 4)

Completing the sort. Notice how the final element resolves itself.

Pass 3	1 2 7 3 9 | 1 2 3 7 9	Search [7, 3, 9]. Min value is 3. Swap 7 and 3.
Pass 4	1 2 3 7 9 | 1 2 3 7 9	Search [7, 9]. Min value is 7 (Itself). Swap 7 with 7.

[1 2 3 7 9]

Final State: The N-1 Rule

We stopped at Pass 4 (n-2). Why? Because if n-1 elements are sorted, the final remaining element (9) is mathematically guaranteed to be in its correct, final position.

Writing the Algorithm

The mechanics of Selection Sort rely on dual nested loops.

```
SelectionSort(Array, n):
  FOR i from 0 to n-2:
    min_index = i

    // Inner Loop: Find Minimum
    FOR j from i+1 to n-1:
      IF Array[j] < Array[min_index]:
        min_index = j

    // Swap Phase
    Swap(Array[i], Array[min_index])
```

The Outer Boundary: i stops at n-2 because the last element naturally falls into place.

The Assumption: We temporarily assume the first unsorted element is the minimum, then prove or disprove it.

The Exhaustive Search: The inner loop j strictly compares the assumed minimum against every remaining element.

The Single Swap: Executes only once per outer loop iteration, minimizing write operations to memory.

Time Complexity Analysis

Why Selection Sort is uncompromisingly O(n^2).

Step 1: Outer loop runs $n-1$ times. Inner loop runs $n-1-i$ times.

Step 2: Total operations summation: $\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$

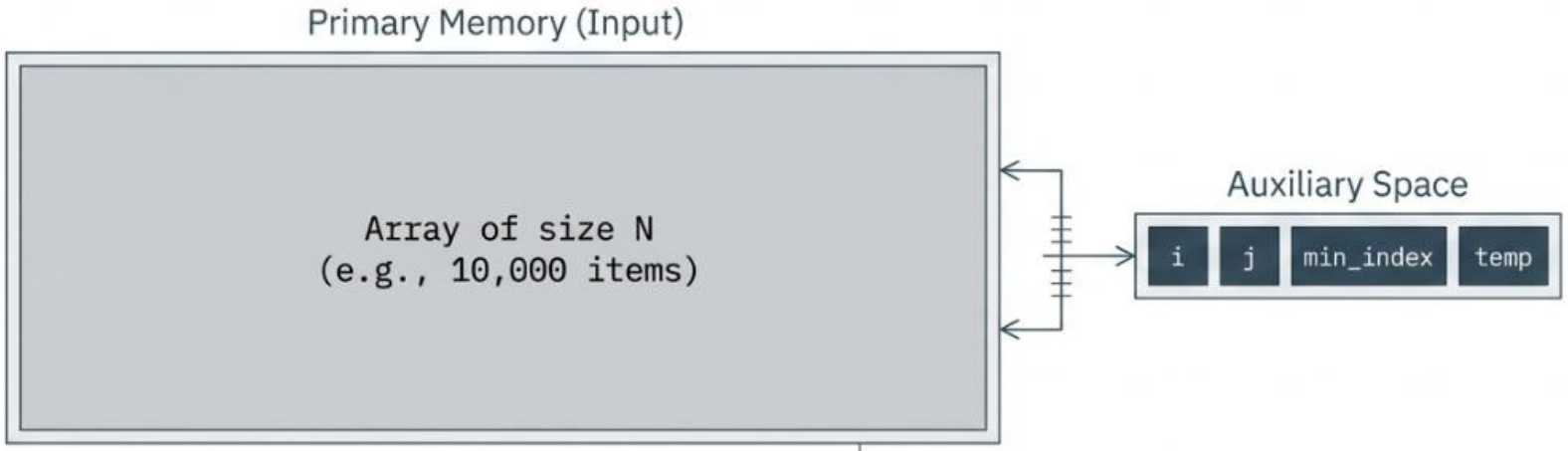
Step 3: Resolves to arithmetic progression: $(n-1)+(n-2)+\ldots+1$

Final Equation: $\frac{n(n-1)}{2} \approx O(n^2)$

Worst Case (Reversed Array)	Average Case (Random Array)	Best Case (Already Sorted)
$O(n^2)$	$\Theta(n^2)$	$\Omega(n^2)$
Must execute both loops fully. Max comparisons, max swaps.	Half sorted/half unsorted inputs still require exhaustive scanning of the unsorted region.	**Crucial Insight:** Unlike other algorithms, Selection Sort lacks a short-circuit condition. Even if given [1, 2, 3, 4], it blindly scans remaining elements every pass to verify the minimum.

Space Complexity

A highly memory-efficient, constant-space algorithm.



Auxiliary Space = O(1)

The algorithm requires a fixed, constant number of variables (roughly 3 to 4) regardless of whether the input array has 10 items or 10,000 items. It does not create new arrays, duplicates, or complex data structures in memory.

Algorithmic Properties

Evaluating memory boundaries and the preservation of order.

In-Place Execution

YES (In-Place)

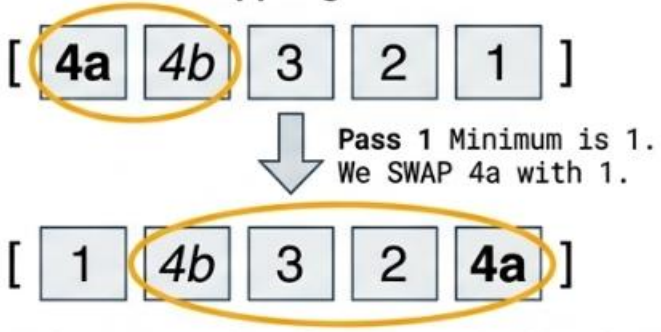
Analysis

Because the Auxiliary Space is O(1), the algorithm rearranges the data directly within the original array boundaries. It requires no external memory blocks or shadow arrays.

Stability

NO (Not Stable)

The Swapping Problem



Insight: The original relative order has been destroyed. 4a now permanently appears after 4b.

To make **Selection Sort stable**, we must abandon swapping. Instead, we find **the minimum and shift all preceding elements to the right**, then **insert the minimum**. (Note: This drastically increases write operations).

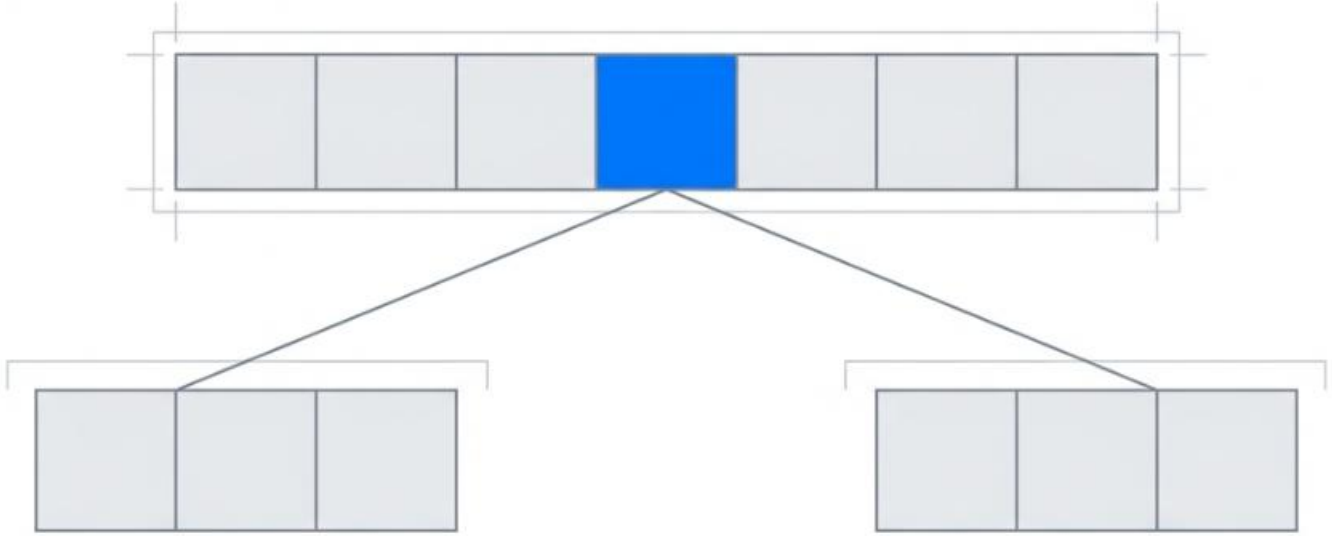
Lecture 8

Sorting Algorithms: Merge - Quick

SORTING ALGORITHMS

3. Quick Sort

A Divide & Conquer Approach to Efficient Sorting



The Philosophy of Quick Sort

1. Select a Pivot

Choose a benchmark number from the array. For our logic, we consistently select the last element.

2. Partition

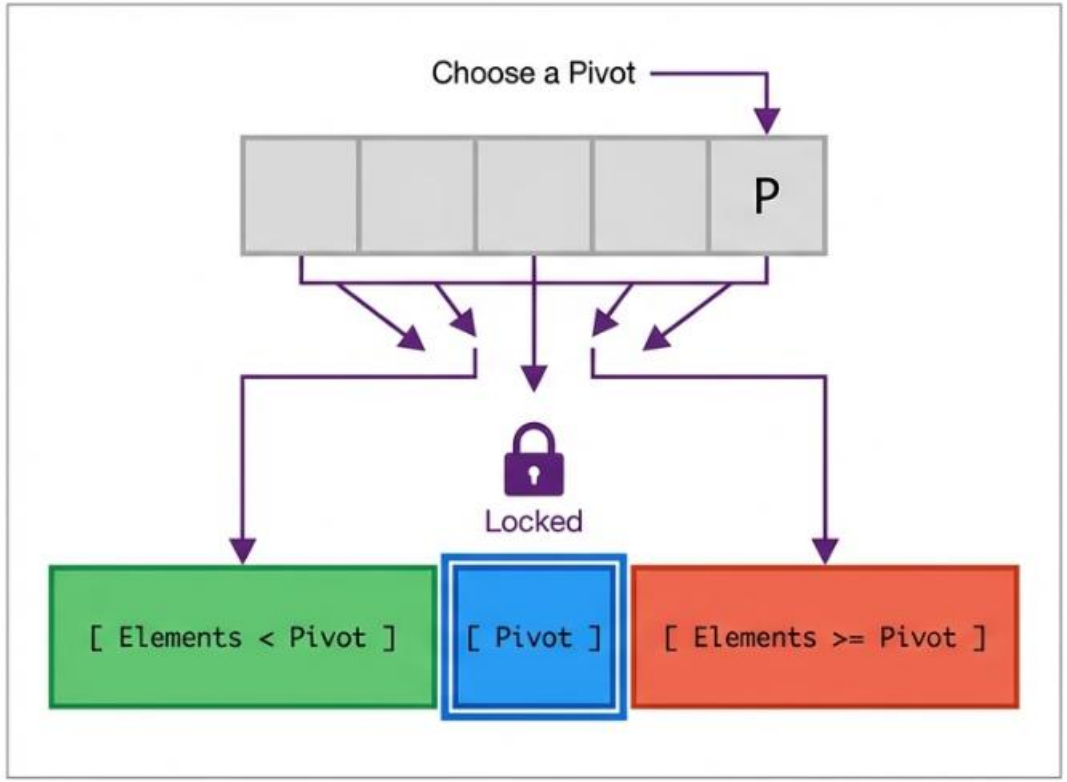
Rearrange the remaining elements. Everything smaller than the pivot is thrown to the left; everything larger to the right.

3. Lock in Place

By isolating smaller elements to the left and larger to the right, the pivot is forced into its absolute, final sorted position.

4. Recurse

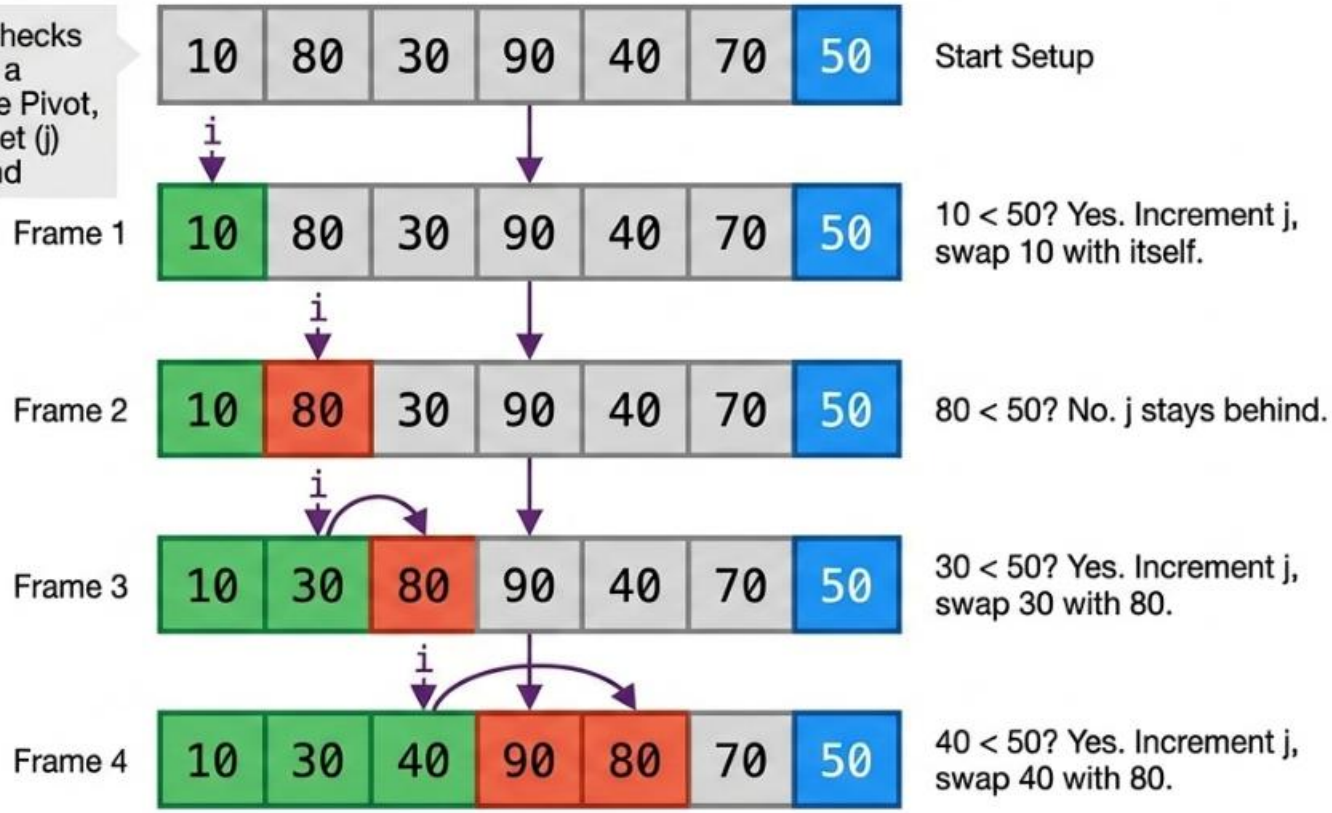
Blindly repeat this process on the left and right fragments until the entire sequence is locked.



The Partitioning Phase: Step-by-Step

< Pivot
Pivot
> Pivot
Pointers

The Golden Rule: As i checks each element, if it finds a number smaller than the Pivot, we move our swap target (j) forward by one step, and swap the elements.

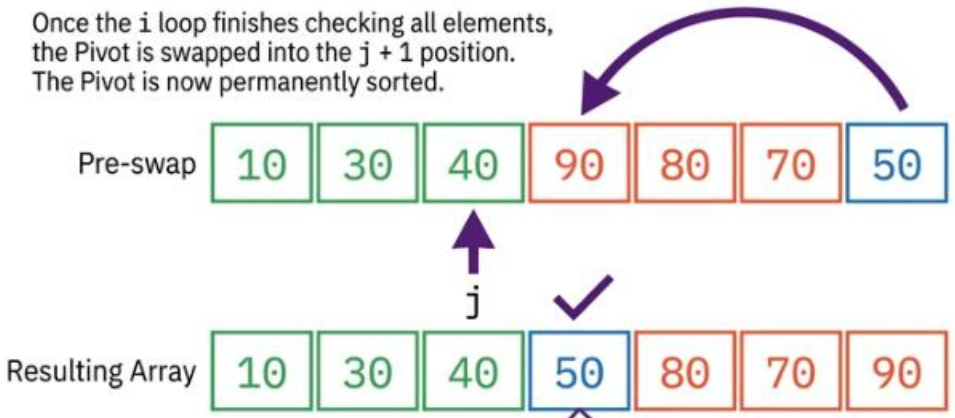


Finalizing the Partition & Recursive Splitting

The Final Swap

Step 1: Place the Pivot

Once the `i` loop finishes checking all elements, the Pivot is swapped into the `j + 1` position. The Pivot is now permanently sorted.



The Recursion Tree

Step 2: Divide & Conquer

The algorithm blindly calls itself on the two newly created sub-arrays until it reaches base cases.

The Algorithm Blueprint

```
QuickSort(arr, low, high):
  if low < high:
    pi = Partition(arr, low, high)

    QuickSort(arr, low, pi - 1)
    QuickSort(arr, pi + 1, high)
```

The Base Case: Stops the recursion when a sub-array has only 1 element (meaning it is natively sorted).

Find the Pivot Index: Executes the sorting logic and returns the locked pivot's final index (pi).

```
Partition(arr, low, high):
  pivot = arr[high]
  j = low - 1

  for i from low to high - 1:
    if arr[i] < pivot:
      j++
      swap(arr[i], arr[j])

  swap(arr[j + 1], arr[high])
  return j + 1
```

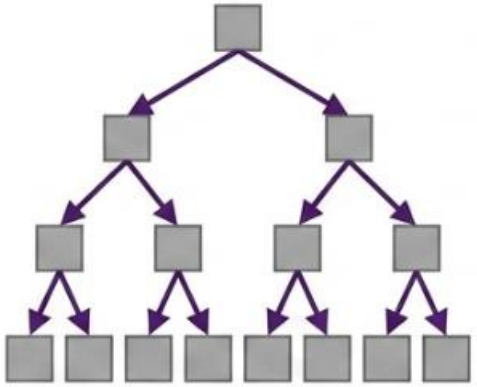
The j Pointer: Tracks the boundary of the "smaller elements" zone.

The Check & Swap: The exact iteration we traced. If an element is smaller than the pivot, expand the small zone (j++) and throw the element into it.

Time Complexity: O, Ω, Θ

Best Case – $\Omega(N \log N)$

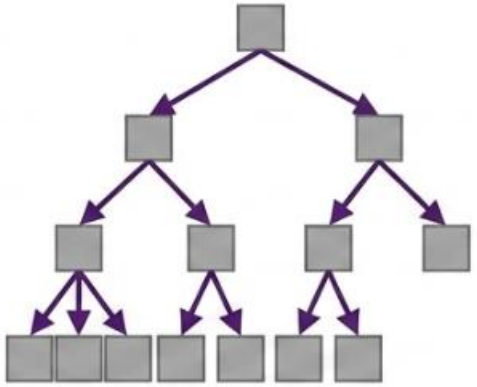
$T(N) = 2T(N/2) + O(N)$



The pivot consistently divides the array into two perfectly equal halves.

Average Case – $\Theta(N \log N)$

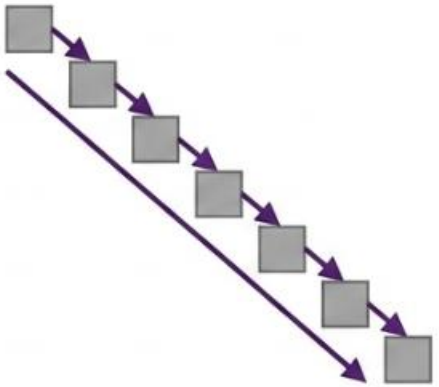
Unbalanced splits (e.g., 1/10 and 9/10)



Even with highly uneven partitions, the overall recursion tree depth remains logarithmic.

Worst Case – $O(N^2)$

$T(N) = T(N-1) + O(N)$



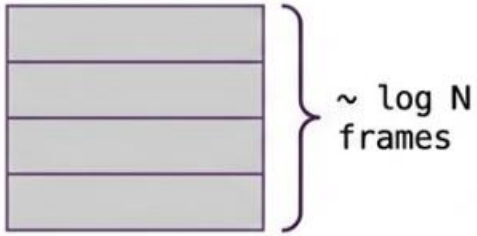
Occurs when the array is already sorted (ascending or descending). One partition is consistently empty, forcing N recursive steps.

Diagnostic Insight: Never use Quick Sort to check if an array is already sorted. Ironically, feeding it sorted data triggers its absolute worst-case performance (N^2).

Space Complexity: The Auxiliary Stack

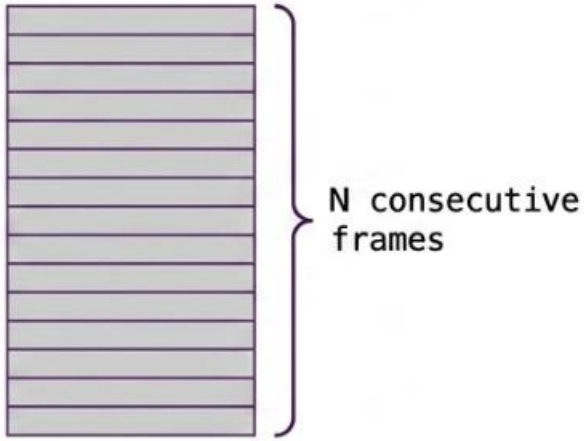
Quick Sort is an "In-Place" algorithm—it rearranges numbers within the original array without creating duplicate arrays. However, it still consumes auxiliary memory through the recursive Call Stack.

Best / Average Space: O(log N)



In a balanced scenario, the maximum depth of recursive functions waiting in memory at any one time is logarithmic.

Worst Space: O(N)

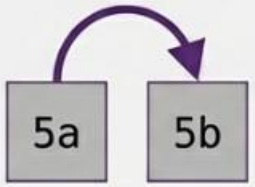


In the worst-case scenario (already sorted data), the recursion tree becomes a straight line. The algorithm builds up N consecutive memory frames before a single one resolves, threatening a Stack Overflow.

Algorithm Benchmarks: Quick Sort vs. Selection Sort

	Is it In-Place?	Is it Stable?
Quick Sort	✔ **Yes.** Sorts entirely within original boundaries using `O(1)` extra data space (excluding the call stack).	✘ **No.** Swapping elements across long distances disrupts the relative order of duplicate items.
Selection Sort	✔ **Yes.** Finds the minimum and swaps it directly, requiring only `O(1)` extra data space.	✘ **No.** Similar to Quick Sort, long-distance swaps of the minimum element can cause equal elements to jump out of their original sequence.

Understanding Stability:
A stable algorithm guarantees that duplicate items retain their original relative input order. Because both Quick Sort and Selection Sort rely on non-adjacent, long-distance 'jumping' swaps, neither is stable by default.



Instability: 5b jumps over 5a

Questions

Goal: Sort the array `[3, 9, 1, 7, 2]`

In insertion sort algorithm we make **comparison and shift** numbers if comparison succeed.
Apply insertion sort algorithm to sort this array,
and write how many comparison & shift occurred to sort it?

Based on the following array, apply the selection sort and state how many swap operation executed?

Answer the following questions

- Worst case of Insertion Sort = ……
- Best case of Insertion Sort = ……
- Average case of Insertion Sort = ……
- Space complexity for Insertion Sort = ……

- Worst case of Selection Sort = ……
- Best case of Selection Sort = ……
- Average case of Selection Sort = ……
- Space complexity for Selection Sort = ……

- Insertion sort is: (a) in-place (b) out-of-place
- Selection sort is: (a) in-place (b) out-of-place
- Insertion sort is: (a) stable (b) not stable
- Selection sort is: (a) stable (b) not stable

Answer the following questions

- Worst case of Merge Sort = ……
- Best case of Merge Sort = ……
- Average case of Merge Sort = ……
- Space complexity for Merge Sort = ……

- Worst case of Quick Sort = ……
- Best case of Quick Sort = ……
- Average case of Quick Sort = ……
- Space complexity for Quick Sort = ……

- Merge sort is: (a) in-place (b) out-of-place
- Quick sort is: (a) in-place (b) out-of-place
- Merge sort is: (a) stable (b) not stable
- Quick sort is: (a) stable (b) not stable

Write and implement the following Algorithms

- Insertion sort
- Selection sort
- Merge sort
- Quick sort

1. **[Your Task]** Prepare presentation for the following sorting algorithms, including (idea, example, algorithm, time and space complexity, when to use this algorithm):
   - **Heap sorting**
   - **Count sort**
   - **Radex sort**